HARNESS LAB · MÓDULO 1 · AULA 1.1

Você virou *operador do agente*

Você usa o agente o dia inteiro. E ainda opera cada passo. O modelo não precisa de mais inteligência: precisa de trilho. Antes de construir o trilho, você mede o seu operador.

`/insights` `/diagnostico-operador` `plugin` `toque de operador` `trecho autônomo` `harness`

VOCÊ SAI COM

O seu "antes" medido em `diagnostico-operador.md`, na raiz do repositório em que você mais usa o agente, pronto para comparar no dia 7.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

O ciclo de quem trabalha com IA o dia todo: pede, sai errado, cola o erro, pede para continuar, refaz. Amanhã, de novo. E muita gente faz isso em três projetos, três repositórios, três features ao mesmo tempo, até perder o controle.

O dev sem padrão e sem harness virou a telefonista dos tempos antigos: fica de olho em tudo, pluga, aprova, troca. Se para para um café, nada anda. A frase que mais ouvimos foi "como usar a IA da maneira certa, sem ficar refazendo as coisas". E a mais dura: "a maioria é vibecoder disfarçado".

Aja como tech lead da sua IA

Tech lead não lê linha por linha. Ele confia em testes automatizados, pipeline de CI, observabilidade, e olha por cima se a entrega cobre a regra de negócio. Revisar 100% do que o agente escreve é humanamente impossível.

Operador x trilho

O operador cola o erro, pede para trocar três linhas, revisa tudo na mão. No trilho, o erro volta sozinho, o agente segue sem você pedir, e um *gate* (teste ou outra comprovação) diz se terminou.

Trocar de modelo não resolve

Pular de provider a cada lançamento é sintoma de operador: usar o modelo de ponta com um arcabouço de pouca autonomia. Estacione num provider e construa o melhor harness para ele.

O modelo não precisa de mais inteligência

Precisa de trilho. Sem trilho ele até chega lá, mas você empurra o trem na mão. Em 7 dias você constrói o seu; no dia 7, o loop entrega a sua feature.

Na demo, o diagnóstico teve dois passos. Primeiro o `/insights`, nativo do Claude Code, que gerou um relatório HTML aberto no navegador. Depois a skill `diagnostico-operador` com o argumento `antes`. Rodada numa pasta que não era projeto, ela perguntou qual projeto medir.

> *"Generate an HTML report analyzing your recent sessions on this machine: which projects you work in, how you use Claude Code, where things go wrong, and features to try."*
>
> Documentação oficial do Claude Code · Commands · /insights

0,9 Toque de operador por entrega no projeto do Lucas, na demo: 8 toques em 9 entregas, 4 sessões, 76 minutos ativos, 23 mensagens (11 pedidos de trabalho novo). Trecho autônomo típico de 0,2 min, o maior de 134 min.

2 O que a aula *não teve tempo de dizer*

ALÉM DA AULA · ONDE FICA O RELATÓRIO DO /INSIGHTS

O relatório mais recente fica em `~/.claude/usage-data/report.html`, com uma cópia datada de cada rodada na mesma pasta. Ele analisa até 200 sessões que ainda não viu, de todos os projetos desta máquina, e pula as muito curtas. Gasta tokens do seu plano. No WSL, abra no navegador do Windows com `explorer.exe "$(wslpath -w ~/.claude/usage-data/report.html)"`; no macOS, `open`; no Linux, `xdg-open`. É o espelho, diferente a cada vez; a métrica é a skill.

ALÉM DA AULA · RODE NA RAIZ DO REPOSITÓRIO CERTO

A skill mede o repositório git da pasta onde você abriu o Claude Code, junto com as sessões abertas em subpastas dele e a cópia do mesmo repositório aberta pelo `/mnt/c` do WSL. Abriu na home ou em `~/projects`, como na demo? O plugin roda `medir.py projetos`, mostra onde há sessões nos últimos 30 dias e pergunta qual medir. Confira as pastas listadas: sessão de outro projeto distorce tudo.

ALÉM DA AULA · COMO LER OS NÚMEROS

Toques por entrega divide os toques pelos commits da janela (`git log --all` dos últimos 30 dias). Squash ou pouco commit inflam o número: informe as entregas reais (`--entregas N`). **Trecho autônomo** é quanto tempo o agente trabalhou sozinho depois de cada mensagem sua. Mediana baixa é o normal de quem conversa passo a passo; é ela que o trilho aumenta. Pausa de mais de 1 hora encerra o trecho.

ALÉM DA AULA · ANTES DE COLAR NOS COMENTÁRIOS

O Lucas pediu a saída do diagnóstico nos comentários. Poste só os números (as linhas de toques, trechos e papéis). O `diagnostico-operador.md` traz exemplos literais das suas mensagens, que podem ter nome de cliente, caminho interno ou dado sensível.

ALÉM DA AULA · O HISTÓRICO SOME EM 30 DIAS

O Claude Code apaga transcrições com mais de `cleanupPeriodDays` dias, 30 por padrão, sem aviso. A foto do "antes" fica salva em `.diagnostico-operador/`, mas a aula 1.3 e o dia 7 ficam mais ricos se o histórico durar mais. No settings de usuário, vale para todos os projetos (passo 6 da instalação).

ALÉM DA AULA · CODEX, CURSOR E OUTROS AGENTES

Nenhum dos dois tem um `/insights` (conferido no Codex CLI 0.166 e na documentação do Cursor em 02/11/2026). O Codex grava as sessões em `~/.codex/sessions/AAAA/MM/DD/*.jsonl`, e o próprio Codex pode ler e escrever o relatório. O Cursor não documenta onde guarda o histórico: busque as conversas com `Ctrl+K` e junte as suas mensagens num arquivo. Usa Gemini CLI, Copilot ou outro? Pergunte ao próprio agente onde ele grava o histórico e siga um dos dois caminhos. Os pedidos prontos estão em `modelos/insights-em-outro-agente.md`.

3 Instale *o que a aula usou*

Na gravação, a skill foi copiada na mão para `.claude/skills/`. Aqui ela vem como plugin do marketplace do curso: o mesmo código, já com as correções desta versão, disponível em qualquer projeto e atualizável. Esta aula não instala arquivos no seu projeto, então não usa o `instalar.sh`.

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Adicione o marketplace do curso e instale o plugin.

Terminal · em qualquer pasta — claude

```
claude plugin marketplace add beerandcodeteam/harness-lab-materiais
claude plugin install diagnostico-operador@harness-lab
claude plugin list
```

O `claude plugin list` precisa mostrar `diagnostico-operador@harness-lab`. Prefere fazer com uma sessão já aberta? Use o bloco abaixo; o `/reload-plugins` carrega o plugin sem reiniciar.

Dentro do Claude Code — sessão

```
/plugin marketplace add beerandcodeteam/harness-lab-materiais
/plugin install diagnostico-operador@harness-lab
/reload-plugins
```

3 Abra o Claude Code na raiz do seu repositório e tire o espelho.

Use o repositório em que você mais trabalhou com o agente no último mês, não um projeto de exemplo. Leia o relatório com calma antes de seguir.

Dentro do Claude Code · na raiz do repositório — sessão

```
/insights
```

4 Tire a foto do "antes".

Dentro do Claude Code · na raiz do repositório — sessão

```
/diagnostico-operador antes
```

A skill mostra as pastas de sessão que leu, pergunta o número de entregas se o repositório não tiver commits na janela, classifica cada mensagem sua (pedido, testou, retomou, aprovou, repetiu, outro) e pergunta o que mais fez você operar o agente nesta semana. Tudo vai para `diagnostico-operador.md`.

5 **Guarde o "antes".**

O melhor é versionar os dois (`git add diagnostico-operador.md .diagnostico-operador`). Não quer no histórico do projeto? Tire do git sem apagar. O `git rev-parse` acha o arquivo certo também em subpasta e em worktree:

Terminal · na raiz do seu projeto — shell

```
echo .diagnostico-operador/ >> "$(git rev-parse --git-path info/exclude)"
echo diagnostico-operador.md >> "$(git rev-parse --git-path info/exclude)"
```

6 **Opcional: guarde o histórico por mais tempo.**

Dentro do Claude Code — sessão

```
> Ponha "cleanupPeriodDays": 90 no meu ~/.claude/settings.json
```

`plugins/diagnostico-operador/skills/diagnostico-operador/SKILL.md` — instalado pelo plugin · não copie do PDF

```
---
name: diagnostico-operador
argument-hint: antes | depois | comparar | listar | projetos
disable-model-invocation: true
allowed-tools: Bash(python3 ${CLAUDE_SKILL_DIR}/scripts/medir.py *), Read, Write
---
O script é `${CLAUDE_SKILL_DIR}/scripts/medir.py`. Ele só lê o histórico
em `~/.claude/projects/` e escreve apenas em `.diagnostico-operador/`
e `diagnostico-operador.md`, na raiz do repositório medido.
```

`disable-model-invocation: true` faz a skill rodar só quando você digita `/diagnostico-operador`: o Claude não decide medir você por conta própria. `${CLAUDE_SKILL_DIR}` é a pasta da skill dentro do plugin, e repetir a variável no `allowed-tools` deixa o script rodar sem pedir permissão a cada passo. A conta (sessões, trechos, commits) é do script, determinística; o modelo só classifica as mensagens.

> **Os outros modos**, para você saber que existem: `listar testou repetiu` abre a prática da aula 1.3; `depois` e `comparar` fecham o dia 7; `projetos` lista onde há sessões. No plugin, o nome completo é `/diagnostico-operador:diagnostico-operador`; o curto funciona quando nenhuma outra skill usa o mesmo nome (testado no Claude Code 2.1.316; a documentação não descreve esse caso).

4 Arquivos que *reforçam a aula*

`skills/diagnostico-operador/SKILL.md`

As regras de classificação, com exemplos reais de cada papel. Ler antes de rodar evita discutir com o resultado: você sabe o que conta como toque e por que pedido nunca conta.

No repositório: plugins/diagnostico-operador/skills/diagnostico-operador/SKILL.md

`skills/diagnostico-operador/scripts/medir.py`

Um trilho pequeno na prática: a parte mecânica (achar sessões, medir trechos, contar commits) é código que dá sempre o mesmo resultado; o modelo só faz o que exige julgamento.

No repositório: plugins/diagnostico-operador/skills/diagnostico-operador/scripts/medir.py

`modelos/insights-em-outro-agente.md`

O espelho do `/insights` para quem também usa Codex ou Cursor: onde está o histórico de cada um e o pedido pronto para colar.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.1-voce-virou-operador-do-agente/modelos/insights-em-outro-agente.md

`teste/sessoes-falsas.py`

Mostra o formato do `.jsonl` que o Claude Code grava e os casos que o diagnóstico separa: subpasta, `/mnt/c`, `claude -p`, projeto vizinho de nome parecido.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.1-voce-virou-operador-do-agente/teste/sessoes-falsas.py

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
"Nenhuma sessão do Claude Code aberta em ..."	Aberto fora do repositório, ou nunca usado nele.	Abra na raiz certa, ou escolha da lista do `projetos`.
`/diagnostico-operador` não aparece	A sessão já estava aberta quando você instalou, ou o marketplace não foi adicionado.	`/reload-plugins`. Se não resolver, `claude plugin list` no terminal.
Roda a versão antiga, sem `projetos`	Sobrou a cópia manual em `~/.claude/skills/` ou `.claude/skills/`, que ganha do nome curto do plugin.	Apague a cópia manual, ou chame `/diagnostico-operador:diagnostico-operador`.
Toques por entrega vazio ou enorme	Repositório sem git, sem commits na janela, ou com squash.	Responda o número real de entregas quando a skill perguntar (`--entregas N`).
Poucas mensagens no "antes"	Pouco uso nos últimos 30 dias, ou sessões já apagadas.	Use uma semana normal de trabalho e rode de novo; aumente o `cleanupPeriodDays`.
`python3: command not found`	Windows fora do WSL.	Rode no WSL, ou instale o Python 3.
Ler o resultado como nota	Muito toque parece "uso errado".	É o retrato de um trabalho sem trilho.

6 Você sai *com*

- [ ] O relatório do `/insights` lido, em `~/.claude/usage-data/report.html`.
- [ ] `claude plugin list` mostrando `diagnostico-operador@harness-lab`.
- [ ] `diagnostico-operador.md` na raiz do repositório com o "antes": toques por entrega, trecho autônomo típico e maior, e quantas vezes você testou, retomou, aprovou e repetiu.
- [ ] A sua frase sobre o que mais fez você operar o agente nesta semana, no fim do arquivo.
- [ ] `.diagnostico-operador/` guardado (versionado ou em `.git/info/exclude`) para a aula 1.3 e o dia 7.

PROMETIDO NA AULA

- ✓ "No material de apoio eu ensino como fazer isso no Codex, no Cursor, entre outros modelos que você usa, outros providers": seção 2 (Codex, Cursor e outros agentes) e `modelos/insights-em-outro-agente.md`.
- ✓ A skill `diagnostico-operador` "que está nos materiais complementares": plugin `plugins/diagnostico-operador/`, instalação na seção 3.
- ✓ O parâmetro `antes` e as métricas da demo: passo 4 da seção 3 e "Como ler os números" na seção 2.
- ✓ Comparar as métricas daqui a 7 dias: modos `depois` e `comparar` da mesma skill, desde que o "antes" fique guardado (passo 5).
- ✓ Deixar a saída nos comentários: o que postar e o que não postar, na seção 2.

7 *Glossário*

Operador	O modo de trabalhar em que você é a peça que faz cada passo do agente andar: cola erro, manda continuar, aprova, revisa tudo.
Trilho (harness)	A estrutura em volta do agente que guia o trabalho sem você olhar cada passo. É o que o curso constrói, no seu repositório.
Gate	Uma comprovação automática de que a entrega está certa: teste, lint, build, checagem de regra.
Toque de operador	Mensagem sua que fez um trabalho que uma estrutura poderia fazer: testou, retomou, aprovou ou repetiu. Pedido de trabalho novo não conta.
Trecho autônomo	Quanto tempo o agente trabalhou sozinho entre uma mensagem sua e a resposta final dele.
Plugin e marketplace	Plugin empacota skills e agentes; marketplace é o catálogo de onde você instala. O do curso se chama `harness-lab`.

8 Para ir *além*

- Analyze your usage patterns: o que o `/insights` lê, onde grava o relatório e quanto custa.
- Skills: frontmatter, `$ARGUMENTS`, `${CLAUDE_SKILL_DIR}` e quem pode invocar cada skill.
- Discover and install plugins: marketplace, escopos de instalação e os comandos `/plugin` e `claude plugin`.
- cleanupPeriodDays: por quanto tempo o Claude Code guarda as sessões e o que mais ele apaga.

Próxima: *Aula 1.2 · O que acontece quando você aperta Enter*

Um modelo mais esperto resolve isso? Para responder, a gente abre o que roda por trás dos panos entre o seu Enter e a resposta do agente.